Step 2: Express Server with WebSocket

ChatGPT React Frontend and Node.js Server

Start React & Server

The next step is to build the Express server.

1. Start the React app (port 3000)

```
npm install
npm start
```

2. Start the Server app (port 5000)

```
npm install
node index.js   # or: npm start
```

Or start both with `./run.sh` in the `step2` directory.

Changing default port number

1. Change port number in `socketConnection/socketConn.js`.

```
import { io } from "socket.io-client";

// Change the port here (or set REACT_APP_SERVER_URL)
const SERVER_URL = process.env.REACT_APP_SERVER_URL || "http://localhost:5001";

export const connectWithSocketServer = () => {
  socket = io(SERVER_URL);
  ...
};
```

2. Set different port before running `node index.js`.

```
PORT=5001 node index.js # to use other port
# with run.sh: SERVER_PORT=5001 CLIENT_PORT=3001 ./run.sh
```

Server

index.js

We process users' requests using WebSocket; however, we also use Express, because it gives developers full control over how requests are processed and responses are sent.

- HTTP requests and responses
- Routing (e.g., /users, /login)
- Middleware for authentication, logging, validation, etc.

We use WebSocket and Express server.

```
const express = require("express");
const http = require("http");
const cors = require("cors");
const socketServer = require("./src/socketServer");

const app = express();

const server = http.createServer(app);
socketServer.registerSocketServer(server);

app.use(cors());

app.get("/", (req, res) => {
  res.send("Hello server is working");
});
```

We use port 5000 for this Express server.

```
const PORT = process.env.PORT || 5000;

server.listen(PORT, () => {
  console.log(`App started listening at port ${PORT}`);
});
```

src/socketServer.js

```
const { Server } = require("socket.io");

const registerSocketServer = (server) => {
  const io = new Server(server, {
    cors: {
      origin: "*",
      methods: ["GET", "POST"],
    },
  });

  io.on("connection", (socket) => {
    console.log(`user connected ${socket.id}`);
  });
};

module.exports = { registerSocketServer };
```

Client

App.js

```
import { useEffect } from "react";
import Dashboard from "./Dashboard/Dashboard";
import { connectWithSocketServer } from "./socketConnection/socketConn";

function App() {
  useEffect(() => {
    connectWithSocketServer();
  }, []);

  return (
    <div className="App">
      <Dashboard />
    </div>
  );
}
```

- useEffect runs after the component first renders
- The empty array [] tells React: “Run this effect only one time, on component mount.”

socketConnection/socketConn.js

This function connects to the server at port 5000.

- When the server emits "connect" API, the `socket.on("connect")` is invoked.

```
import { io } from "socket.io-client";

let socket;

// Change the port if necessary (run.sh passes REACT_APP_SERVER_URL)
const SERVER_URL = process.env.REACT_APP_SERVER_URL || "http://localhost:5000";

export const connectWithSocketServer = () => {
  socket = io(SERVER_URL);

  socket.on("connect", () => {
    console.log("connected with socket.io server");
    console.log(socket.id);
  });
};
```